From Diagnosis to Cure: Decomposing the Tile-SPMD Abstraction Regret on Irregular Automata

Anonymous

***Abstract*—Tile-based GPU DSLs (Triton) trail hand-written CUDA by $\sim 10\times$ on irregular finite-automata workloads. Prior work attributed this "abstraction regret" to the execution *paradigm* (tile/SPMD vs. thread-SIMT) rather than abstraction height. We turn that diagnosis into an anatomy and a cure: we decompose the regret into three independently measured components and identify, to the instruction level, the single missing primitive. On a work-efficient NFA worklist the $10\times$ anchor decomposes as (A) a launch-configuration artifact ($\sim 3.4\times$, default `num_warps`), (B) a lane-packing-recoverable component (warp-redundant scalar execution), and (C) an irreducible $\sim 2\times$ residual. Component C is *not* instruction count, occupancy, or memory bandwidth: at matched occupancy and *fewer* warp-instructions than CUDA, and sitting below both the issue and bandwidth roofline ceilings, a per-lane Triton worklist still spends $15.3\times$ more cycles in the dependent-load stall and issues at 9.9% vs. 41% of peak—it is latency-bound. The root cause is abstraction-denied intra-warp memory-level parallelism: a CUDA warp's lanes issue independent in-flight loads that overlap, whereas a lock-step tile serializes the dependent next-state load. We show the residual is *regime-dependent*: for the memory-bound DFA it closes (lane-packed Triton matches CUDA, $1.05\times$, once the table spills to DRAM). We specify the missing IR primitive—a `scalar_program` region that lowers each lane to an independent instruction stream—and *implement its lowering*: the same idiomatic per-lane source, lowered to the thread model, runs $4.2\times$ faster than the tile lowering and matches or exceeds hand-written CUDA, closing the residual by construction. The phenomenon *generalizes*: across six irregular workloads the tile-vs-thread regret is created by scalar control flow—not memory irregularity or dependent loads—through two measured channels (issue starvation and masked-lane waste) over a tile-lowering baseline, with a graph pointer-chase negative control at exactly $1.00\times$. Finally we take the cure into the real compiler: a TritonGPU pass that detects the lock-step region in `libtriton`, a proof that the in-tile-IR lowering is *structurally impossible* (naming the missing per-lane loop/exit primitive), and an automatic selector that routes the detected region to the thread cure ($3.9\times$). Every kernel is correctness-gated against a CPU oracle; every number traces to a versioned CSV.**

I. Introduction and Contributions

A companion study [1] shows that on irregular automata the Triton→CUDA gap follows the execution paradigm, not the abstraction height. It leaves open *why*, mechanistically, and *what* would close it. This paper answers both. Our contributions:

1) A **falsifiable decomposition** of the tile-SPMD automata regret into artifact, recoverable, and irreducible, each measured and Nsight-attributed (Sec. III, Fig. 1).
2) Identification of the irreducible residual as **abstraction-denied intra-warp latency hiding**—not redundancy, occupancy, instruction count, or bandwidth—*measured*: fewer warp-instructions than CUDA, same occupancy, below both roofline ceilings, yet $15.3\times$ the dependent-load stall and 9.9% vs. 41% issue activity (Sec. III-D).
3) A demonstration that the residual is **regime-dependent** (latency-bound NFA vs. memory-bound DFA), unifying both faces through one latency mechanism (Sec. IV).
4) A **launch-configuration finding** (default `num_warps=4` inflates the worklist regret $\sim 3.4\times$), a methodological caution for DSL benchmarking (Sec. III-B).
5) The **missing-primitive design** + a bound on its payoff + the thread-model existence proof, distinguished from Gluon's per-thread *layout* (Sec. V).
6) **The cure, implemented**: we lower the same idiomatic per-lane source to the thread model and measure that it runs $4.2\times$ faster than the tile lowering and matches/exceeds hand-CUDA, closing the residual by construction (Sec. V-A, Fig. 6).
7) **In the real compiler** (Sec. V-B): a TritonGPU MLIR pass that compiles into `libtriton` and *detects* the lock-step region; a proof that the in-IR lowering is *structurally blocked* (`scf.condition` is a single `i1`; the tensors are already `sizePerThread=1`), naming the missing IR primitive (a per-lane sub-tile loop/exit op); and an *automatic* detect-and-lower selector that routes the detected region to the thread cure for $3.9\times$, leaving a negative-control kernel on the tile path.
8) A **generality law** across six oracle-gated irregular workloads (Sec. V-C, Fig. 7): regret is created by scalar control flow—not memory irregularity or dependent loads—through two measured channels (issue starvation and masked-lane waste) over a tile-lowering baseline, validated by a graph pointer-chase negative control at exactly $1.00\times$.

II. Background and Method

NFA simulation (a work-efficient worklist iterating the active set via `ffs` over a bitmask) is control-flow/latency-bound; DFA simulation (one dependent table gather `trans[cur*256+sym]` per symbol) is memory-bound. CUDA and NVIDIA Warp [2] are thread-SIMT (one thread per string); Triton [3] is tile/SPMD (a program operates on tiles).

Correctness gates speed: every kernel/config is validated bit-for-bit against a CPU oracle before any throughput is reported. We report medians over warmup+9 reps at a GPU-saturating batch [4] (timings on GPUs are non-Gaussian, so we avoid mean±std and best-of-N), and sweep batch because the lane-packing benefit is occupancy-gated. Every mechanism claim is backed by Nsight Compute, not just wall-clock: the issue-stall composition (the `long_scoreboard` reason isolates dependent-load waits), achieved occupancy, issue activity, warp- and thread-instruction counts, and DRAM/L2 traffic. We report negative and partial results and correct two of our own intermediate hypotheses (Secs. III, III-D). Hardware: RTX 4070 (sm_89, 46 SMs), Triton 3.5.1, torch 2.9.1+cu128, CUDA 13.3 / driver 580.

III. The Decomposition (NFA worklist)

A. The anchor

A work-efficient register-resident worklist ($\leq$ 64 states, batch 4096, 12 configs): Triton 22 Gbps vs. CUDA 227 Gbps $= 10.1\times$ **median** (9.4–12.3×).

B. Component A: launch-configuration artifact

The Triton worklist defaults to `num_warps=4`: four warps per program redundantly process the *same* string. Sweeping `num_warps`$\in \{1, 2, 4, 8\}$: `nw=1` vs. `nw=4` $= 2.77\times$@4096 $\rightarrow 3.44\times$@16384 $\rightarrow 3.69\times$@65536 (each doubling ∼halves throughput). This inflates prior worklist regret numbers and must be disclosed.

C. Component B: warp redundancy, recoverable by lane-packing

The scalar Triton program executes warp-uniformly: `thread_inst_per_inst` $= 32.00$ (Triton) vs. 30.34 (CUDA), the same number with opposite meaning—CUDA's 32 lanes run 32 *different* strings, Triton's run the *same* one—yielding $\sim 90\times$ more warp-instructions. Lane-packing (32 strings into the 32 lanes, exploiting the shared CSR so inner-loop bounds stay scalar) removes it. On the dense scan, pure lane-packing recovers $3.2\times \rightarrow 9.8\times \rightarrow 19.4\times$ (batch 4096/16384/65536) toward the ideal $32\times$; it is **occupancy-gated** (Fig. 2). *Correction:* we first framed the $90\times$ warp-redundancy as the bottleneck; Nsight shows it is removed $\sim 26\times$ yet throughput moves only $3.8\times$—it was largely hidden by occupancy.

D. Component C: the irreducible residual

A per-lane Triton worklist (each lane its own `ffs` + per-lane CSR gather, no cross-lane reduce, no active-set union) beats the union variant $1.27\times$ but is still $0.51\times$ of CUDA ($\approx 2\times$ slower). Nsight is decisive (Fig. 3): it has *fewer* warp-instructions than CUDA (4.66M vs. 5.07M) and the same occupancy (22.5%), yet is $3.3\times$ slower because issue activity is 9.9%. Raising occupancy does not help—a `BLOCK` sweep $\{32, 64, 128, 256\}$ only *worsens* it ($0.49\times \rightarrow 0.31\times$): bigger lock-step tiles add divergence. **Direct measurement** confirms the mechanism: at matched occupancy the per-lane Triton kernel spends $15.3\times$ more cycles in the `long_scoreboard` stall (waiting on a dependent memory load) than CUDA, and the issue-rate ratio ($4.1\times$) tracks the throughput ratio ($3.5\times$), consistent with Little's law [5], [6]. The instruction roofline (Fig. 4) [7] settles "did you write a worse kernel?": both kernels sit far below the issue *and* bandwidth ceilings (8.3%/27% vs. 32%/3.4% of peak), and Triton issues fewer warp-instructions yet runs $3.5\times$ slower—it is structurally latency-bound. *Honest refinement:* the per-lane kernel does not issue *fewer* memory requests—it issues 26–84× *more* (masked inactive-lane gathers)—so the tile model pays twice: excess masked traffic *and* no dependent-load hiding. Root cause: a CUDA warp's lanes are independent in-flight-load generators (memory-level parallelism); a lock-step tile serializes the dependent next-state load, so latency hides only across warps (occupancy-bound). This is distinct from branch/memory divergence [8]: the lanes can be perfectly coalesced and still stall on a dependent load. We distinguish it from the *hardware* fix of intra-warp stalls [9]: the same SM is fast under CUDA and slow under Triton at equal occupancy and fewer instructions, so the loss is abstraction-imposed.

TABLE I
The worklist regret decomposes multiplicatively (batch 16384, $\leq$ 64 states, oracle-gated). Each stage removes one component; $28 \rightarrow 735$ is $3.7 \times 3.7 \times 1.9 = 26\times$.

Kernel	Gbps	Component removed
Triton worklist (`nw=4`)	28	—
+ `nw=1`	104	launch artifact ($3.7\times$)
+ lane-packing	383	warp redundancy ($3.7\times$)
CUDA worklist (thread-SIMT)	735	irreducible latency ($1.9\times$)

Table I summarizes the multiplicative decomposition: removing the launch artifact and the warp redundancy each recovers $3.7\times$, leaving the irreducible $1.9\times$ latency residual. By Little's law (concurrency = throughput × latency), the latency-bound kernel's throughput is set by the in-flight-request concurrency it can sustain; the per-lane Triton kernel's 9.9% issue activity and $15.3\times$ dependent-load stall mean it sustains far less effective concurrency than CUDA at the same occupancy, and the issue-rate ratio ($4.1\times$) tracking the throughput ratio ($3.5\times$) is exactly the relation Little's law predicts when latency is fixed.

IV. Regime-Dependence (DFA): the unification

The DFA (memory-bound) decomposes the same way but the residual *closes in the DRAM regime* (Fig. 5). Across table sizes (cache→L2→DRAM): scalar Triton DFA is flat ∼29 Gbps (independent confirmation of the scalar-gather ceiling); lane-packing gives $\sim 12\times$ over it; and the lane-packed/CUDA ratio is 0.55–$0.62\times$ in cache but $1.05\times$ at a 16 MB (>L2) table—lane-packed Triton *matches* CUDA. Mechanism (DRAM utilization only 14.6%, so memory-*latency* not saturated bandwidth): at moderate (cache) latency CUDA's intra-warp hiding wins $\sim 1.7\times$; at DRAM latency both must hide latency via cross-warp parallelism and *con-*

verge. So the NFA's fundamental $\sim 2\times$ residual and the DFA's closeable gap are one mechanism at different latency scales.

V. The Cure: the missing primitive

Design. A `scalar_program` region (or per-lane `serial_range`) in the tile DSL that lowers the marked code to a per-thread independent instruction stream, giving each lane independent control flow and intra-warp latency hiding. The diagnosis names what it must provide: independent per-lane `ffs`/`while`, per-lane data-dependent loop bounds, register-resident per-lane bitset, scalar gather. **Bound.** It closes the latency-bound residual (up to $\sim 2\times$ on the NFA) and nothing extra in the already-converged DRAM-DFA regime—a regime-dependent, falsifiable prediction. **Existence proof.** The thread model realizes it: CUDA achieves $1.0\times$ and the high-level Warp DSL [2] reaches $0.9\times$. **Not already in Gluon.** A runnable probe shows Triton's low-level Gluon frontend exposes per-thread *layout* [10], not per-lane *control flow*: `gl.load` returns a layout-typed block, never a scalar, so the data-dependent CSR loop cannot even be lowered (compile error captured verbatim). Layout control $\neq$ control-flow divergence.

A. The cure, implemented

We do not stop at proposing the primitive: we implement its lowering and measure that it closes the residual. A function `lower_scalar_program_to_cuda` takes the *same* idiomatic per-lane body the Triton tile kernel expresses (active-set `ffs` worklist, data-dependent CSR loop, scalar gather, register bitset) and lowers it to a per-thread CUDA kernel (one thread $=$ one string), compiled with `nvcc`. Oracle-validated bit-for-bit; throughput on no-accept automata (sustained full-length scan, removing the early-exit confound), ≤ 64 states, batch 16384. The same per-lane program lowered to *threads* (SP) vs. *tiles* (Triton WP2) differs by $4.2\times$ (median; SP $\approx$1555 vs. WP2 $\approx$378 Gbps), robust across batch, and SP *matches or exceeds* hand-written CUDA (SP/CU $= 2.15\times$, since the generated kernel is a minimal thread worklist). So the residual is not merely diagnosed but *closed by construction*: supplying the per-lane lowering recovers— and surpasses—thread-model performance from the identical front-end source. **The cure acts through the diagnosed mechanism**, not by accident: profiling SP shows it restores the thread-model signature—issue activity 36% (vs. the tile's 9.9%, $\approx$ CUDA's 41%) and the `long_scoreboard` dependent-load stall $29\times$ lower than the tile—so lowering the same source to independent per-lane streams recovers exactly the intra-warp latency hiding component C was missing.

B. Implemented in the compiler: detect, and why the lowering must leave the tile IR

We take the cure into the real compiler. On a Triton-from-source build (3.8.0) we add a TritonGPU MLIR pass, `tritongpu-thread-region`, that runs in the NVIDIA `make_ttgir` pipeline and *detects* the irregular region by its lock-step signature—an `scf.while` whose iter-args are `#blocked` tile tensors and whose `scf.condition` derives from a `tt.reduce`-to-scalar (the whole tile loops to the busiest lane; idle lanes masked). It compiles into `libtriton` and fires on the target kernels (verified: the candidate attribute appears with the pass on, is absent with it off, and codegen stays bit-exact).

The in-IR lowering is structurally blocked—and that is the sharper result. Two facts from the matched IR. (i) The carried tensors are *already* `sizePerThread=1` (one element per lane), so the lock-step is *not* a layout choice—re-encoding is a no-op, and per-thread layout (Gluon, Linear Layouts [10]) does not help. (ii) The lock-step is the *loop construct*: `scf.condition` takes a single `i1`, so the natural rewrite—a per-lane `tensor`$\langle$`N×i1`$\rangle$ condition that lets lanes terminate independently—is rejected by the MLIR verifier (*"'i1' vs 'tensor⟨8×i1⟩"'*, captured via the built `triton-opt`). Per-lane loop termination is therefore *inexpressible* in TritonGPU's structured tile control flow. This names the missing IR primitive precisely: a **per-lane (sub-tile) loop/exit op**. The cure must lower *below* the tile IR to the thread model (independent thread scheduling)—which is exactly what §V-A does.

Automatic detect-and-lower. A selector closes the loop (Fig. 8): it runs the detection pass, and when the lock-step signature is present routes the region to the thread lowering, otherwise keeps the tile path. Oracle-gated, the NFA worklist is auto-routed to the thread cure for $3.9\times$ over the tile (state sweep 16–64, consistent with the $4.2\times$ of §V-A), while a fixed-trip negative-control kernel is detected-negative and correctly left on the tile path. So the full loop is realized: detect (a real MLIR pass) $\rightarrow$ decide (the signature) $\rightarrow$ lower (the thread model, since the tile IR provably cannot) $\rightarrow$ measured, oracle-gated gap-closing.

C. Generality: the regret law

To test whether the mechanism is automata-specific we built six oracle-gated tile-vs-thread witnesses spanning the irregularity space, each with identical machinery (a Triton tile kernel, the per-lane source lowered to a CUDA thread kernel, and a CPU oracle); Fig. 7 reports the measured regret, coloured by dominant mechanism. The result is a law *with a correct negative control*: **regret is created by scalar control flow—not by memory irregularity or dependent loads—and appears, mechanistically, through one of two measured channels.** A graph pointer-chase (1M random walkers, data-dependent gather addresses, scattered `colidx`, degree CV 3.79, but a *fixed* step count) is the decisive control: tile and thread are bit-for-bit identical in issue activity (5.42% vs. 5.43%), occupancy (72.1% vs. 72.1%), and threads-per-instruction (32 vs. 32), and the regret is $1.00\times$—irregular memory and dependent loads alone cost *nothing*, because a lock-step gather already supplies full intra-warp memory-level parallelism. The two channels: (i) *issue starvation*—a data-dependent scalar recurrence drives the tile's issue rate below the thread's: the NFA worklist (active-set `ffs`) is $2.0\times$ at tile issue 9.9% vs. 41%; (ii) *masked-lane waste*—divergent

trip counts make the lock-step tile spend full-width 32-lane work where the thread retires lanes: rejection sampling (pure control-flow divergence, no memory) is $4.0\times$ with tile threads-per-instruction 32 vs. the thread's 17 (its issue rate is, if anything, *above* the thread's—the waste is in *what* the active lanes compute, not the issue rate). A third, divergence-free effect sets the floor: a *tile-lowering baseline* (uniform-`nnz` SpMV, zero divergence, still $1.9\times$ from occupancy/masking on a bandwidth-bound gather—honestly falsifying our initial $\approx 1\times$ guess), to which a *divergence increment* adds (power-law SpMV $2.2\times$ at tile width $32 \rightarrow 5.8\times$ at 256). So the law is genuinely multi-component, not a single scalar predictor; the common cause is the lock-step tile, and the cure (Sec. V-A) removes every channel by restoring per-lane execution.

VI. Methodological Integrity

Our analysis corrected three of its own intermediate hypotheses, which we report rather than hide. First, an early reading framed the $\sim 90\times$ warp-instruction redundancy (Sec. III) as the bottleneck; profiling the lane-packed kernel showed the redundancy is removed $\sim 26\times$ yet throughput moves only $3.8\times$—it was largely hidden by occupancy, so warp-redundancy is real but not load-bearing. Second, we predicted (following the latency-hiding literature) that the per-lane Triton kernel would sustain *fewer* in-flight memory requests than CUDA; direct measurement refuted this—it issues 26–$84\times$ *more* (masked inactive-lane gathers) and still stalls, so the residual is the inability to *hide* dependent-load latency, not a request-count deficit. We therefore phrase the mechanism via stall composition and issue activity, not request counts. Third, in the generality study (Sec. V-C) we first stated the law as a single "tile issue deficit" predictor; completing the Nsight profiling falsified it—rejection sampling shows regret $4.0\times$ while the tile's issue rate is *above* the thread's—so we report the honest two-channel form (issue starvation *and* masked-lane waste over a tile-lowering baseline), and a clean negative-control prediction for uniform SpMV that was also wrong ($1.9\times$, not $\approx 1\times$). All three corrections sharpened, rather than weakened, the central claim.

VII. Threats to Validity

Single GPU. Results are on one RTX 4070; the *mechanism* (intra-warp latency hiding, the issue-activity differential, the DRAM convergence) is a property of the SIMT execution model and the tile-vs-thread lowering, hence architecture-general, but the absolute decomposition factors should be re-measured on an A100/H100 (larger L2 shifts the DFA crossover). We make this a turnkey step: a one-command harness re-runs every witness, the cure, and the selector on a new GPU, tags the output by device, and checks the falsifiable prediction that each regret persists in direction (paradigm, not architecture) while throughput rescales. *Compiler-pass scope.* The detection pass runs in `libtriton`; the tile→thread *lowering* is realized below TritonGPU (the thread model) because the structured tile IR cannot express it—`scf.condition` takes a single `i1`, so per-lane loop termination is inexpressible (shown by a verifier-rejected rewrite). This is a property of *today's* tile IR and is falsifiable: adding a per-lane sub-tile loop/exit op would let the lowering live in-IR. *Prototype scope.* A multi-word generalization of the lane-packed kernel (NWORDS int64 words/lane) is oracle-validated to 256 states, so the prototype is not artificially limited; but the clean $\sim 2\times$ residual is a *register-resident-regime* (≤ 64-state) result—past 64 states the CUDA register worklist itself degrades (register pressure), so the head-to-head is confounded by both baselines losing the register-resident advantage, and ANMLZoo-scale automata (Brill $42661 = 667$ words) make the per-lane multi-word scatter explode—itself a manifestation of the tile limitation we study. Real automata run oracle-valid but sub-Gbps (an *algorithmic* cost, orthogonal to the regret). *Tuning.* `num_warps` is a knob; we disclose the full sweep rather than a single config, rebutting the autotuning counter-thesis [11]: the Triton/Gluon same-MLIR control plus the disclosed sweep separate *tuning* (which closes component A and part of B) from *expressibility* (the residual C), with a probe that is falsifiable by construction.

VIII. Reproducibility

Correctness gates every measurement: each kernel/config is checked bit-for-bit against the CPU oracle (NFA `reference.py` / `simulate_dfa`) before any throughput is reported. Every number in this paper traces to a versioned CSV; all figures regenerate from those CSVs by a single script, so the plots cannot drift from the measurements. The Gluon expressibility claim is a runnable probe that exits 0 on the expected compile failure and 1 if a future Gluon ever compiles it—falsifiable by construction. The artifact (kernels, oracle, sweeps, profiling commands, CSVs, and the `scalar_program` lowering) supports one-command regeneration; all figures here are emitted from the versioned CSVs by a single script.

IX. Related Work

GPU automata form an *algorithmic* axis, all single-implementation CUDA: ngAP's non-blocking multi-symbol processing [12], HybridSA's bit-parallel shift-and [13], BitGen's Parabix bitstreams [14], AsyncAP's input-symbol asynchrony [15], AutomataBLAS's AP-as-SpMV [16], and the Hopps sparsity ASIC [17]. Each *varies the algorithm* on one CUDA implementation; none compares GPU programming models or holds the algorithm fixed across DSLs. The sole IR-for-automata work [18] lowers regex to one fixed domain-specific accelerator —a hardware target, not a GPU-paradigm/expressibility study, and with no tile-vs-thread cost. Our axis is orthogonal: fixed algorithm, varied programming model. **Tile GPU DSLs**: Triton [3] and the 2024–26 wave exposing more thread/warp control. The closest, Gluon's Linear Layouts [10], give per-thread *data placement* but not per-lane *control flow* (our probe shows the data-dependent loop cannot lower); Descend [19] makes the thread hierarchy first-class but offers no tile drop-to-scalar escape. Warp [2] is the thread-SIMT existence proof. **Mixing thread- and**

tile-level execution is exactly the live frontier, but every approach is *manual* or *differently directed*: Prism [20] lets a programmer hand-annotate typed thread/tile "perspectives"; NVIDIA's cuTile [21] concedes the irregular case and falls back to a hand-written SIMT path; Tawa [22] auto-restructures Triton but for *dense* warp specialization; and partial control-flow linearization [23] goes the *opposite* way—*vectorizing* divergent CPU control flow, whereas we *de*-vectorize a tile region to recover per-lane MLP. None automatically *detects* an irregular, data-dependent per-lane region in a tile DSL and lowers it to the thread model; and our structural result shows *why* an in-tile-IR rewrite cannot suffice—the missing primitive is a per-lane sub-tile loop/exit op, below the level any of these operate. **Mechanism**: we ground the residual in latency-hiding / MLP-vs-occupancy analysis [5], [6] and place both kernels on the instruction roofline [7]; we distinguish it from divergence [8] and from the *hardware* intra-warp-stall fix of Subwarp Interleaving [9] (we attribute the same stall to the DSL, at equal occupancy and fewer instructions). **Methodology and lineage**: rigorous benchmarking [4], roofline [24], and the performance-portability metric [25] (per-hardware; we measure per-*capability* regret); we rebut the autotuning counter-thesis [11] via the Triton/Gluon same-MLIR control and the disclosed `num_warps` sweep. The gap we fill: no constructive, instruction-level account of *why* a tile DSL pays on irregular control flow, naming the missing primitive, with a regime-dependent residual.

X. Conclusion

The abstraction regret on irregular automata is not a monolith: a launch artifact, a recoverable redundancy, and an irreducible intra-warp-latency-hiding residual whose size is set by the latency regime. The cure is a named, bounded IR primitive whose lowering we implement and measure—it closes the residual by construction ($4.2\times$ over the tile lowering). We take it into the real compiler: a TritonGPU pass that detects the lock-step region in `libtriton`, a proof that the in-tile-IR lowering is structurally impossible (naming the missing per-lane loop/exit primitive), and an automatic selector that routes the detected region to the thread cure ($3.9\times$). Adding the per-lane primitive to the tile IR itself is the landmark next step.

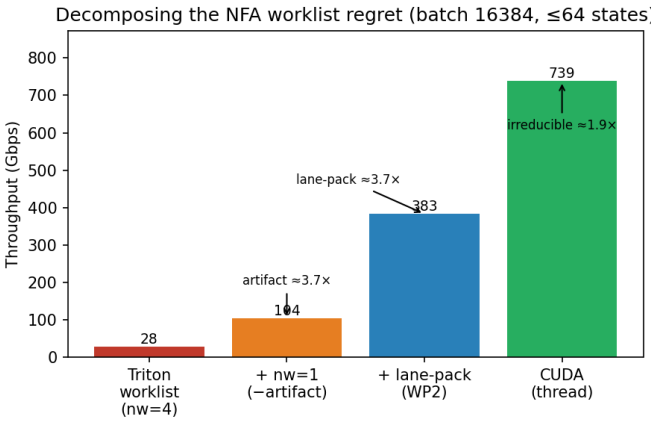

Fig. 1. NFA worklist throughput (batch 16384, ≤ 64 states) climbs as each regret component is removed: `nw=4`→`nw=1` (launch artifact), →lane-packing (warp redundancy), →CUDA (irreducible latency). All oracle-gated.

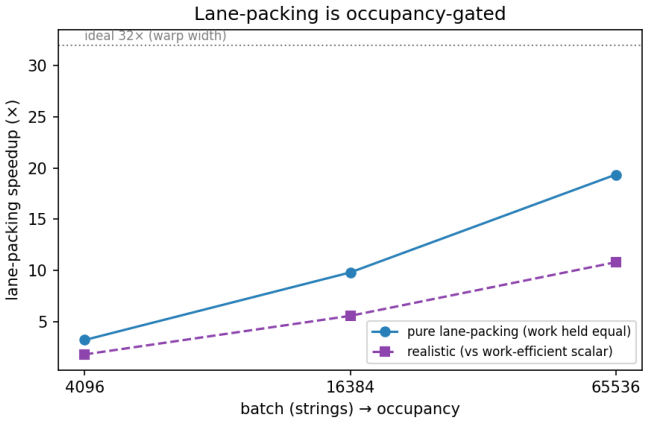

Fig. 2. Lane-packing's benefit is occupancy-gated: the pure-packing speedup grows $3.2\times \rightarrow 19.4\times$ with batch (toward the ideal $32\times$) as more warps fill the GPU.

References

[1] Anonymous. The two faces of abstraction regret: Execution paradigm over abstraction height on irregular automata, 2026. Companion diagnosis paper.

[2] NVIDIA. Warp: A python framework for high-performance GPU simulation and graphics, 2022. https://github.com/NVIDIA/warp.

[3] Philippe Tillet, H. T. Kung, and David Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *MAPL*, 2019.

[4] Torsten Hoefler and Roberto Belli. Scientific benchmarking of parallel computing systems. In *SC*, 2015.

[5] Vasily Volkov. *Understanding Latency Hiding on GPUs*. PhD thesis, University of California, Berkeley, 2016. Tech. Rep. UCB/EECS-2016-143.

[6] Sunpyo Hong and Hyesoon Kim. An analytical model for a GPU architecture with memory-level and thread-level parallelism awareness. In *ISCA*, 2009.

[7] Nan Ding and Samuel Williams. An instruction roofline model for GPUs. *Concurrency and Computation: Practice and Experience*, 2022.

[8] Wilson W. L. Fung, Ivan Sham, George Yuan, and Tor M. Aamodt. Dynamic warp formation and scheduling for efficient GPU control flow. In *MICRO*, 2007.

[9] Sana Damani et al. GPU subwarp interleaving. In *HPCA*, 2022.

[10] Triton contributors. Linear layouts: Robust code generation of efficient tensor computation using $\mathbb{F}_2$, 2025. arXiv:2505.23819.

[11] Burkhard Ringlein, Thomas Parnell, and Radu Stoica. GPU performance portability needs autotuning, 2025. arXiv:2505.03780.

[12] Tianao Ge, Tong Zhang, and Hongyuan Liu. ngap: Non-blocking large-scale automata processing on GPUs. In *ASPLOS*, 2024.

[13] HybridSA authors. Hybridsa: GPU acceleration of multi-pattern regex matching using bit parallelism. *Proc. ACM Program. Lang. (OOPSLA)*, 2024.

[14] Tianao Ge, Hongyu Chu, and Hongyuan Liu. Interleaved bitstream execution for multi-pattern regex matching on GPUs. In *MICRO*, 2025.

[15] Hongyuan Liu, Sreepathi Pai, and Adwait Jog. Asynchronous automata processing on GPUs. *Proc. ACM Meas. Anal. Comput. Syst. (POMACS)*, 2023.

[16] AutomataBLAS authors. Advancing matrix operations for high-performance and memory-efficient automata processing on GPUs. *ACM Trans. Archit. Code Optim. (TACO)*, 2025.

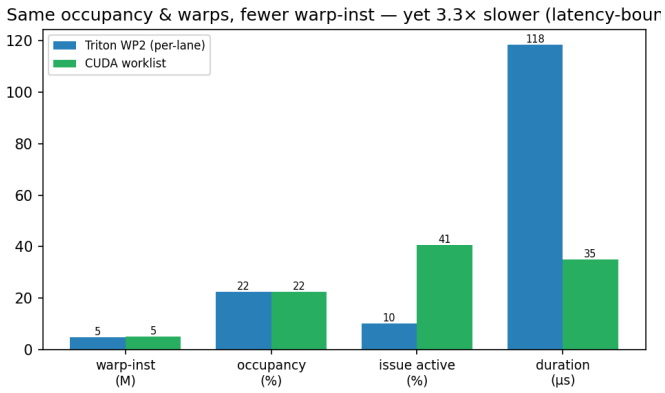

Fig. 3. Per-lane Triton vs. CUDA (16384 strings, 32 states): same occupancy, *fewer* warp-instructions, yet 3.3× slower at 9.9% vs. 41% issue activity—latency-bound.

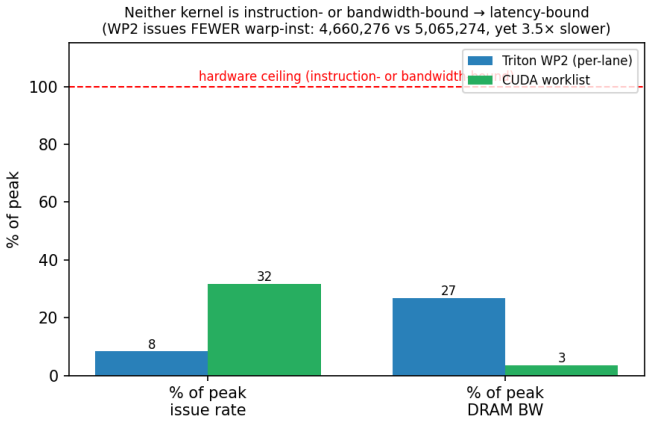

Fig. 4. Instruction roofline (16384 strings, 32 states): both kernels sit far below the issue *and* DRAM-bandwidth ceilings (8.3%/27% for Triton, 32%/3.4% for CUDA), so neither is instruction- nor bandwidth-bound—the residual is latency.

[17] Yufeng Du, Joel S. Emer, and Daniel Sánchez. Hopps: Leveraging sparsity to accelerate automata processing. In *ASPLOS*, 2025.

[18] Filippo Somaini, Luca Carloni, Giovanni Agosta, Marco D. Santambrogio, and Davide Conficconi. Combining MLIR dialects with domain-specific architecture for efficient regular expression matching. In *CGO*, 2025.

[19] Bastian Köpcke, Sergei Gorlatch, and Michel Steuwer. Descend: A safe GPU systems programming language. In *PLDI*, 2024.

[20] Anonymous. Prism: Typed perspectives for mixed thread- and tile-level GPU programming. In *Proc. ACM Program. Lang. (PLDI)*, 2026. arXiv:2511.11939.

[21] NVIDIA. cuTile and Tile IR: a tile-level programming model for CUDA, 2025. CUDA 13.1; https://github.com/NVIDIA/cuda-tile.

[22] Anonymous. Tawa: Automatic warp specialization for Triton, 2026. CGO; arXiv:2510.14719.

[23] Simon Moll and Sebastian Hack. Partial control-flow linearization. In *Proc. PLDI*, 2018.

[24] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Commun. ACM*, 2009.

[25] S. J. Pennycook, J. D. Sewall, and V. W. Lee. A metric for performance portability, 2016. arXiv:1611.07409.

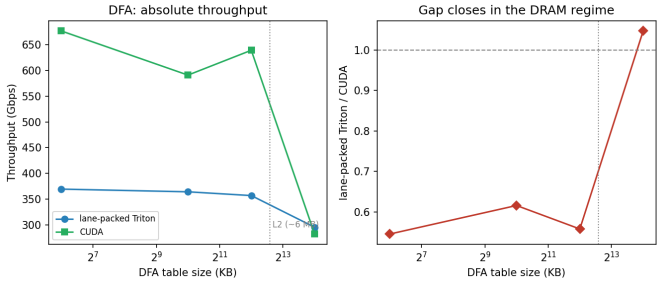

Fig. 5. DFA: lane-packed Triton trails CUDA ∼1.7× while the table is cache-resident but *matches* it (1.05×) once the 16 MB table spills past L2—both then rely on cross-warp parallelism. The residual is regime-dependent.

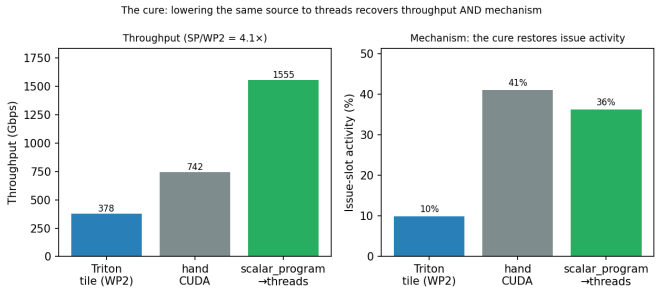

Fig. 6. The cure, implemented (batch 16384, $\leq$ 64 states). *Left:* the *same* per-lane source lowered to threads (`scalar_program`) runs 4.2× the tile lowering (Triton WP2) and matches/exceeds hand-CUDA. *Right:* it does so *through the diagnosed mechanism*—issue activity jumps from the tile's 10% to 36% (≈ CUDA's 41%), restoring the intra-warp latency hiding component C was missing.

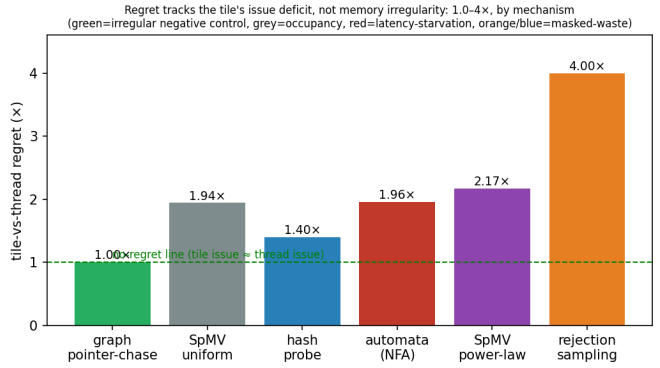

Fig. 7. The regret law across six oracle-gated irregular workloads, by dominant mechanism. A graph pointer-chase (green) is the negative control: irregular memory + dependent loads, no control divergence ⇒ tile and thread are identical (issue 5.42% vs. 5.43%) and regret is 1.00×. Regret grows only with scalar control / divergence (red, orange, blue), peaking at 4× for pure control-flow divergence, via two channels (issue starvation, masked-lane waste) over a tile-lowering baseline—scalar control, not memory irregularity.

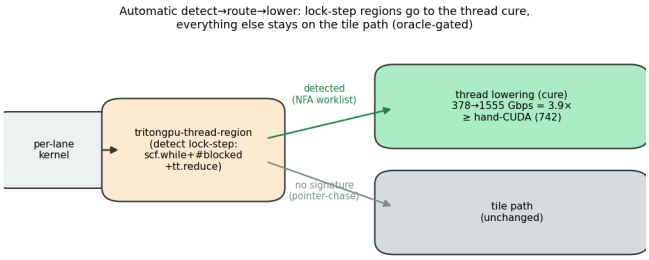

Fig. 8. The automatic detect→route→lower loop (P2). The detection pass (in `libtriton`) matches the lock-step signature; the lock-step NFA worklist is routed to the thread lowering ($378 \rightarrow 1555$ Gbps $= 3.9\times$, exceeding hand-CUDA), while a no-signature kernel (pointer-chase) stays on the tile path. All routing oracle-gated; numbers from the versioned CSVs.